Lab 08 — CPU Timers

Clock configuration, 1 kHz timer polling and 4 kHz sine-wave sampling

TMS320F28379D | Code Composer Studio (CCS) | Duration: 2 hours

Objective

You will investigate the CPU timer clock, modify the supplied CPU Timer1 configuration to produce a 1 kHz timing event, generate a numerical 4 kHz sine-wave lookup table, and use Timer1 polling to collect and export 100 samples. The experiment demonstrates the consequences of undersampling.

Project: Lab 07 CPU Timer starter project (CPU_Timers_01.c). The sine wave is represented by numbers in memory; this lab does not generate an analogue voltage or use an ADC.

Lab schedule

Activity	Time
Part 1 — Examine source project and clock	20 min
Part 2 — Change CPU Timer1 to 1 kHz and verify	20 min
Part 3 — Generate 4 kHz sine-wave table	20 min
Part 4 — Modify while(1) and acquire 100 samples	35 min
Part 5 — Export, plot and explain results	25 min
Total	120 min

Part 1 — Examine the source project and system clock (20 minutes)

Step 1: Inspect the supplied program

1. Import the supplied project into CCS and open CPU_Timers_01.c.

2. Identify Device_init(), initCPUTimers(), configCPUTimer(), CPUTimer_startTimer() and the while(1) loop.

3. Identify the original periods of CPU Timer0 and CPU Timer1.

4. Open device.c and locate SysCtl_setClock(DEVICE_SETCLOCK_CFG). Follow DEVICE_SETCLOCK_CFG to its definition in device.h.

5. Open the CCS Registers view and inspect CLKSRCCTL1, SYSPLLCTL1, SYSPLLMULT and SYSCLKDIVSEL.

Q1. What are the original CPU Timer0 and CPU Timer1 intervals, in seconds?

Q2. What is the difference between the oscillator frequency and SYSCLK? What is the purpose of the PLL?

Q3. Which oscillator source, PLL multiplier and SYSCLK divider are selected in the supplied project? Calculate SYSCLK.

Clock parameter	Value from code / CCS
Oscillator source	
Oscillator frequency	
PLL multiplier	
SYSCLK divider	
Calculated SYSCLK	

Part 2 — Configure CPU Timer1 for 1 kHz (20 minutes)

Before modifying the sampling loop, you must change CPU Timer1 from its original 2 s interval to an interval of 1 ms. CPU Timer0 should remain at 0.5 s.

Step 2: Modify and verify Timer1

1. Locate the configCPUTimer() calls in main().

2. Keep the CPU Timer0 configuration unchanged. Change only the third argument of the CPU Timer1 call to give a 1 ms period. Note that the function accepts a period in microseconds, not a frequency in hertz.

3. Build and debug the project. Confirm that CPU Timer1 is running.

4. In the CCS Registers view, examine CPU Timer1 PRDH:PRD (period), TIMH:TIM (counter), TPR/TPRH (prescaler) and TCR (control).

5. Record the configured period and check the TSS bit. Do not change the timer period while running without reloading the counter.

Q4. What is the period in microseconds corresponding to a 1 kHz timer frequency?

Q5. What numerical third argument is required in configCPUTimer() for CPU Timer1?

Q6. Assuming SYSCLK = 200 MHz and prescaler = 0, calculate the requested number of clock cycles in 1 ms. Compare it with the value written to PRDH:PRD by the supplied function.

Q7. Does the CPU timer count up or down? Which TCR bit starts or stops it?

Timer setting	Original	Modified / observed
CPU Timer0 interval	0.5 s	
CPU Timer1 interval	2 s	
CPU Timer1 event frequency	0.5 Hz	
CPU Timer1 PRDH:PRD		
CPU Timer1 prescaler		

Part 3 — Generate a 4 kHz sine-wave lookup table (20 minutes)

Step 3: Build the numerical waveform

Create a floating-point array sineTable[32] containing one complete cycle of a sine wave. Use the expression below to populate the array once, before entering while(1).

$x[n] = \sin(2\pi n / 32), \quad n = 0, 1, \ldots, 31$

1. Include math.h and define a suitable constant for π.

2. Write a function generateSineTable() to fill sineTable[32].

3. Call the function once before starting the acquisition.

4. Inspect the table in the CCS Expressions or Memory Browser view.

Q8. What is the period of a 4 kHz sine wave, in microseconds?

Q9. What is the notional waveform-generation rate for 32 points per 4 kHz cycle?

Q10. What are the expected values at sineTable[0], [8], [16] and [24]?

Note: The table defines samples of a 4 kHz waveform. It is not continuously output by the microcontroller. During acquisition, students must calculate which phase of this waveform would be present at each 1 ms timer event.

Part 4 — Sample the waveform using CPU Timer1 (35 minutes)

Step 4: Modify the while(1) loop

1. Declare a float array acquired[100] and a sample counter initialised to zero.

2. Modify while(1) to poll the CPU Timer1 overflow/interrupt flag (TIF), rather than simply reading the countdown register. Do not enable timer interrupts.

3. When Timer1 expires, clear its TIF flag using the appropriate DriverLib function, then store exactly one sine-wave value in acquired[].

4. Advance the sine-wave phase by the number of table positions corresponding to 1 ms elapsed. Wrap the index modulo 32.

5. After 100 samples, stop writing to acquired[]. Halt the debugger when ready to export the array.

Pseudocode (students must implement the C statements):

```
while (1) {
  if (Timer1 expiry flag is set) {
    clear Timer1 expiry flag;
    if (sampleCount < 100) {
      acquired[sampleCount] = sineTable[tableIndex];
      advance tableIndex by the correct phase increment;
      sampleCount++;
    }
  }
}
```

Q11. How many complete cycles of a 4 kHz sine wave occur during a 1 ms interval?

Q12. How many positions must the 32-element table index advance per Timer1 event? What is that increment modulo 32?

Q13. If the first sample is taken at zero phase, predict all 100 acquired values. Explain the result.

Q14. Why is reading CPUTimer_getTimerCount() repeatedly inside while(1) not equivalent to sampling at 1 kHz?

Part 5 — Export and analyse the acquired samples (25 minutes)

Step 5: Inspect, save and plot

1. Build and run the program until sampleCount reaches 100.

2. Inspect acquired[100] using CCS Expressions or Memory Browser.

3. Use the CCS memory-export/save facility to save the array to a separate file on the host PC (for example acquired_1kHz.dat). Record the start address, data type and export format. If the output is hexadecimal or binary, convert it to numeric text before plotting.

4. Plot the acquired samples against time t[n] = n × 1 ms using MATLAB, Python or another plotting tool.

5. Compare the plot with the intended 4 kHz sine wave and explain any difference.

Q15. What does the acquired waveform look like? Does it reproduce a 4 kHz sine wave?

Q16. State the Nyquist sampling requirement and explain why 1 kHz is insufficient for a 4 kHz sine wave.

Q17. What sampling frequency would be required to represent a 4 kHz sine wave without aliasing in principle? Why is a rate strictly above 8 kHz preferable?

Q18. What happens if the software takes longer than 1 ms to service a Timer1 expiry?

Quantity	Calculated / observed
SYSCLK	
Timer1 interval	
Timer1 event frequency	
Sine frequency and period	
Sine-table size	
Table phase advance per timer event	
Number of acquired samples	
Expected / observed waveform	

Submission

Submit: (1) the modified C source code; (2) screenshots of the Timer1 clock and timer registers; (3) the exported acquired-data file; (4) a plot against time; (5) completed tables; and (6) answers Q1–Q18. State clearly that the acquisition uses polling rather than interrupts.

Optional extension

Change CPU Timer1 to a sampling rate of 16 kHz and repeat the acquisition. Calculate the new timer interval and table-index increment, then compare the acquired waveform with the 1 kHz result. Do not modify CPU Timer0.

Practical note: A timer overflow flag indicates that at least one expiry occurred; it does not count multiple missed periods. The debugger may alter observed timer behaviour depending on the TCR FREE and SOFT settings.